PUC Minas

CONCEITOS E FUNDAMENTAÇÃO

OBJETIVO

Compreender os principais conceitos primários e fundamentais que envolvem Desenvolvimento, Segurança e Operações

Por que DevSecOps e não SecDevOps?

PUC Minas

DESENVOLVIMENTO (DEV)

PROGRAMA

Um **programa** é uma **entidade** que representa uma **máquina abstrata** que se manifesta de duas formas: (1) como um ***documento***, ele especifica uma sequência de operações a serem executadas; (2) durante a sua ***execução***, ele efetivamente leva a cabo as operações especificadas

Fonte: MELO, Ana Cristina Vieira. Et al. Princípios de Linguagens de Programação. 2022. Blucher. São Paulo.

SOFTWARE

Software consiste em: (1) instruções (programas de computador) que, quando executadas, fornecem características, funções e desempenho desejados; (2) estruturas de dados que possibilitam aos programas manipular informações adequadamente; e (3) **informação descritiva**, tanto na forma **impressa quanto na virtual**, **descrevendo a operação e o uso dos programas**.

Fonte: PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9 ed. 2021. Pag. 5. Grupo A. Porto Alegre/RS.

ENGENHARIA DE SOFTWARE

Engenharia de software: a aplicação de uma abordagem **sistemática**, **disciplinada** e **quantificável** no desenvolvimento, na operação e na manutenção de software; isto é, a aplicação de engenharia ao software.

Fonte: PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9 ed. 2021. Pag. 5. Grupo A. Porto Alegre/RS.

ENGENHARIA DE SOFTWARE



Fonte: PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9 ed. 2021. Grupo A. Porto Alegre/RS. Camadas da Engenharia de Software.

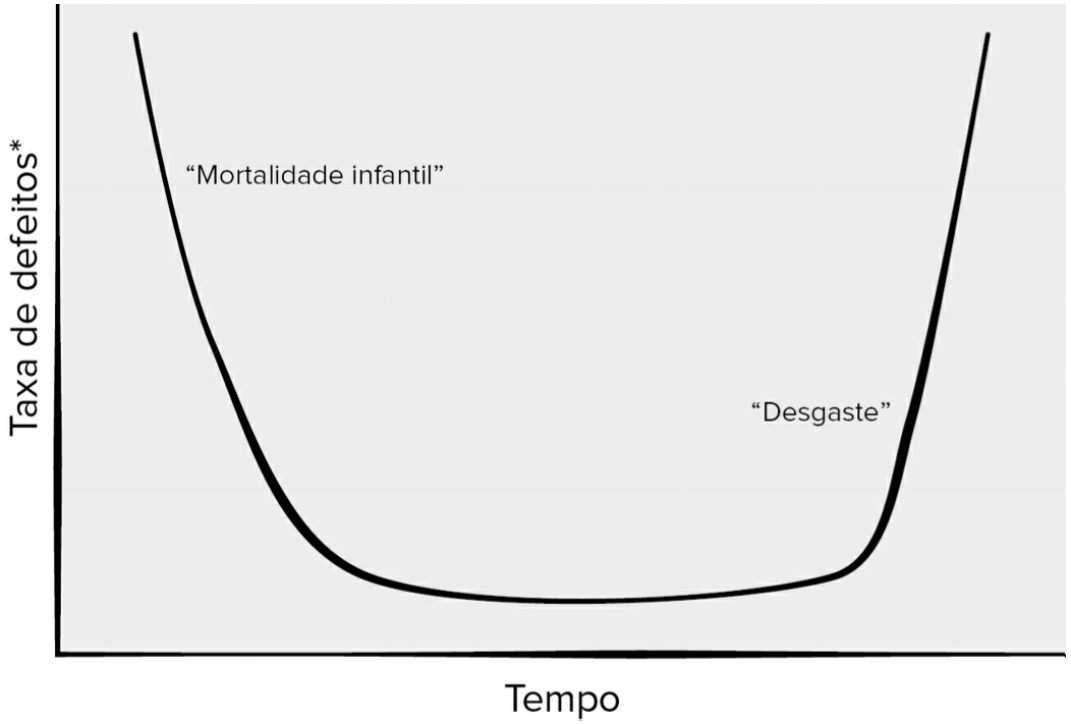
ENGENHARIA DE SOFTWARE

- Engenharias Tradicionais
  - Engenharia Civil, Engenharia Elétrica, Engenharia Mecânica
- Modelos de engenharia foram aplicados à engenharia de software, portanto:
  - **Planejamento detalhado**
  - **Execução Sequencial**

ENGENHARIA DE SOFTWARE

- Engenharia de Software # Engenharia Tradicional
- Software # Automóveis, Ponte, Casa, Avião
- **Software # Produto Físico/Hardware**
- Software é Abstrato
- Software é **Adaptável**
- **Hardware** se **desgasta**, **software** se **deteriora**

ENGENHARIA DE SOFTWARE

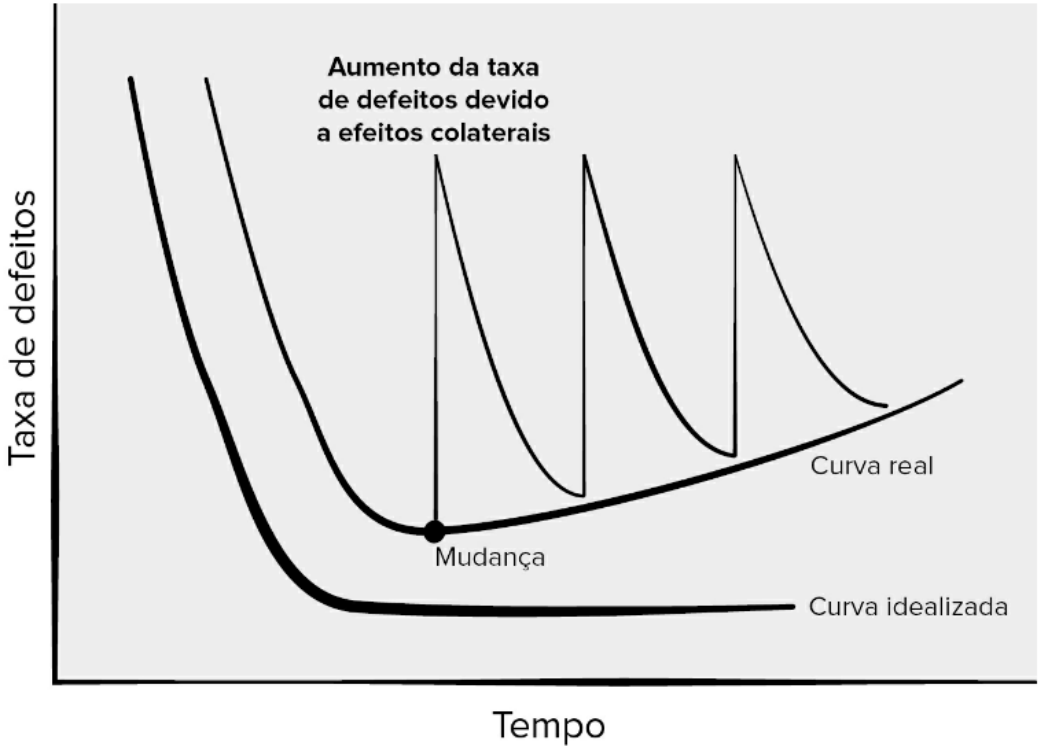


Taxa de defeitos em função do tempo para **hardware**.

Relação, normalmente denominada "curva da banheira". O hardware apresenta taxas de defeitos relativamente altas no início de sua vida (defeitos de projeto ou de fabricação); os defeitos são corrigidos, e a taxa cai para um nível estável por certo período. À medida que o tempo passa, a taxa aumenta novamente, conforme os componentes de hardware sofrem os efeitos cumulativos de poeira, vibração, impactos, temperaturas extremas e vários outros fatores maléficos do ambiente. Resumindo, o **hardware começa a se desgastar**.

Fonte: PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9 ed. 2021. Grupo A. Porto Alegre/RS.

ENGENHARIA DE SOFTWARE



O **software** não é suscetível aos fatores maléficos do ambiente que causam o desgaste do hardware. Portanto, teoricamente, a curva da taxa de defeitos para software deveria assumir a forma da "curva idealizada". Defeitos ainda não descobertos irão resultar em altas taxas logo no início da vida de um programa. Entretanto, esses defeitos serão corrigidos, e a curva se achata. **Software não se desgasta.**

Mas deteriora!

Fonte: PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9 ed. 2021. Grupo A. Porto Alegre/RS.

DESENVOLVIMENTO DE SOFTWARE: HISTÓRICO



- Ada Lovelace (1815-1852) é considerada a primeira programadora de computador do mundo
- Criou um algoritmo para calcular os números de Bernoulli, usando a máquina analítica de Charles Babbage
- Elaborou planos de como a máquina Analytical Engine deveria realizar cálculos

Fonte: https://pt.m.wikipedia.org/wiki/Ficheiro:Ada_Lovelace_portrait.jpg

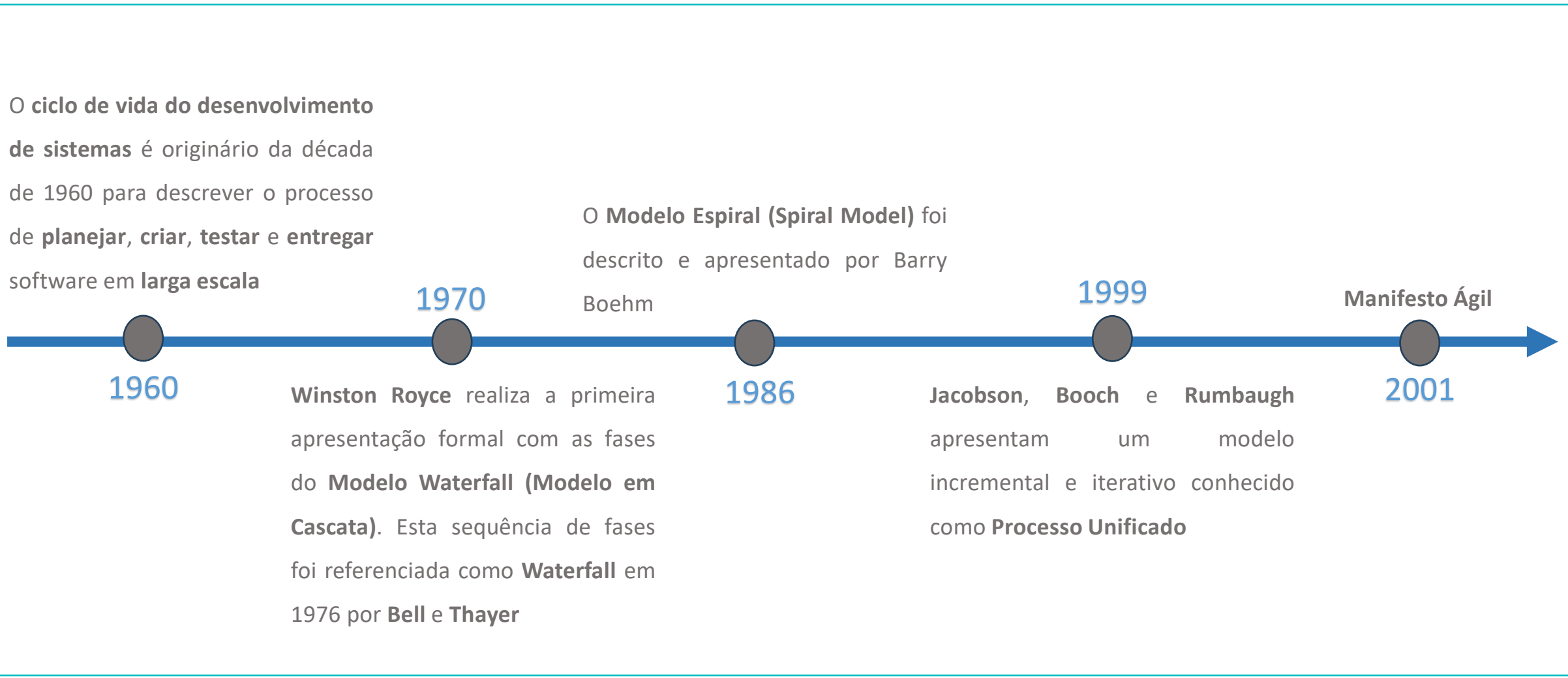
DESENVOLVIMENTO DE SOFTWARE: HISTÓRICO



- Em 1948, Tom Kilburn e Freddie Williams desenvolveram um programa de computador para a execução de cálculos
- O programa foi armazenado em memória eletrônica, no computador Manchester Baby
- Tom Kilburn também desenvolveu o Ferranti Mark I, o primeiro computador comercial

Fonte: https://pt.m.wikipedia.org/wiki/Ficheiro:Ada_Lovelace_portrait.jpg

MODELOS DE DESENVOLVIMENTO DE SOFTWARE: HISTÓRICO



PUC Minas

O SOFTWARE E A SOCIEDADE

- Softwares possuem impactos consideráveis na sociedade
  - Netflix, Uber, Amazon, iFood, mídias sociais, IA
- A popularidade da Internet e potencialização de dispositivos móveis (consequentemente, Big Data) e as redes sociais (**Nexus das Forças**, Gartner) impactaram na ascendência vertiginosa da **produção de softwares** e **novos modelos de negócio**

O SOFTWARE E A SOCIEDADE

1970 - 1980	1990	2000	2010 - Presente
• **Era**: Mainframe • **Tecnologia**: Cobol, DB2 • **Ciclo**: 1 – 5 anos • **Custo**: $1 mi - $100 mi • **Risco**: empresa inteira • **Custo da falha**: falência, venda da empresa, demissão em massa	• **Era**: Client/Server • **Tecnologia**: C++, Oracle • **Ciclo**: 3 – 12 meses • **Custo**: $100 mil - $10 mi • **Risco**: uma linha de produto • **Custo da falha**: perda de lucro, troca de CIO	• **Era**: Nuvem • **Tecnologia**: Java, MySQL • **Ciclo**: 2 – 12 semanas • **Custo**: $100 mil - $1 mi • **Risco**: um recurso do produto • **Custo da falha**: insignificante	• **Era**: Mobile e IA • **Tecnologia**: Fluter, Reactive, MongoDB • **Ciclo**: 2 – 4 semanas • **Custo**: $10 mil - $1 mi • **Risco**: uma feature do produto • **Custo da falha**: insignificante

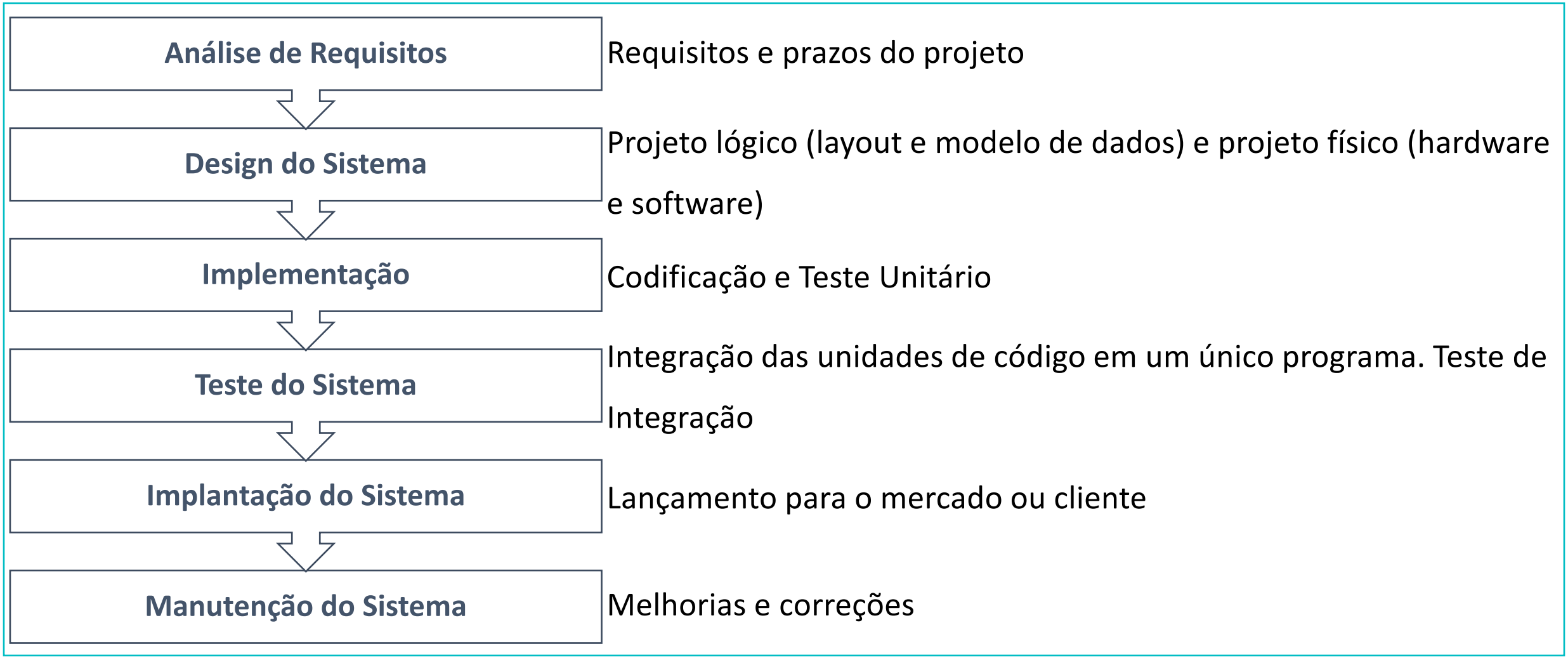
Fonte: MUNIZ, Antônio. Jornada Ágil do Produto. 2020. Brasport. A **infraestrutura** se tornou **acessível**, o **ciclo de desenvolvimento** mais **ágil** e **eficiente**, **menos arriscado** e **menos custos**.

PUC Minas

PUC Minas

MODELO WATERFALL (CASCATA)

DESENVOLVIMENTO: MODELO WATHERFALL



ENGENHARIA DE SOFTWARE

De forma ampla, **Requisitos** definem o que um sistema deve fazer e sob quais restrições. O **Requisitos Funcionais** representam o que o sistema deve fazer, enquanto, **Requisitos Não-Funcionais**, representa sob quais restrições, que são as características que ele deve ter para funcionar bem

Segurança é um **Requisito Funcional** ou um **Requisito Não-Funcional**?

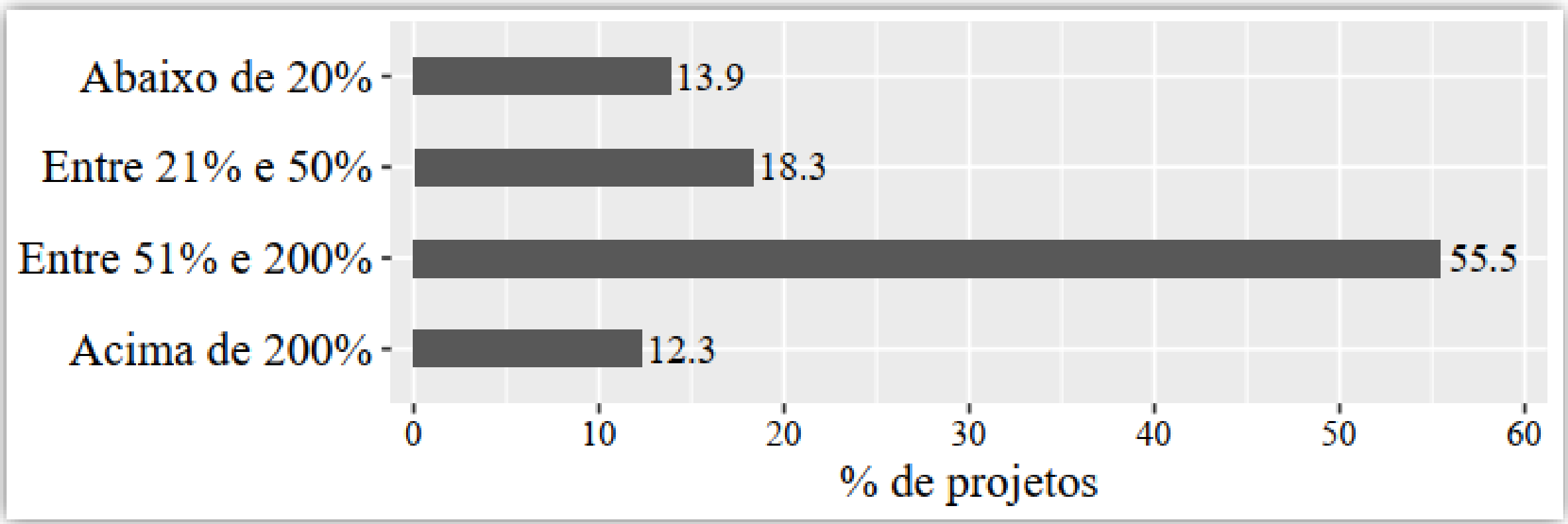
MODELO WATHERFALL: DESVANTAGENS

- É um **modelo de alto risco**, tornando-o **impróprio para projetos complexos** e **orientados a objetos**
- É muito difícil fazer alterações no código quando o desenvolvimento da aplicação chega à fase de testes
- Atrasa a entrega do produto porque **não são criados protótipos intermediários** e o produto é entregue no final do ciclo

DESENVOLVIMENTO: MODELO WATHERFALL

Caso o cliente fique insatisfeito com o produto, o desenvolvimento do produto deverá ser reiniciado, resultando em atraso e custos

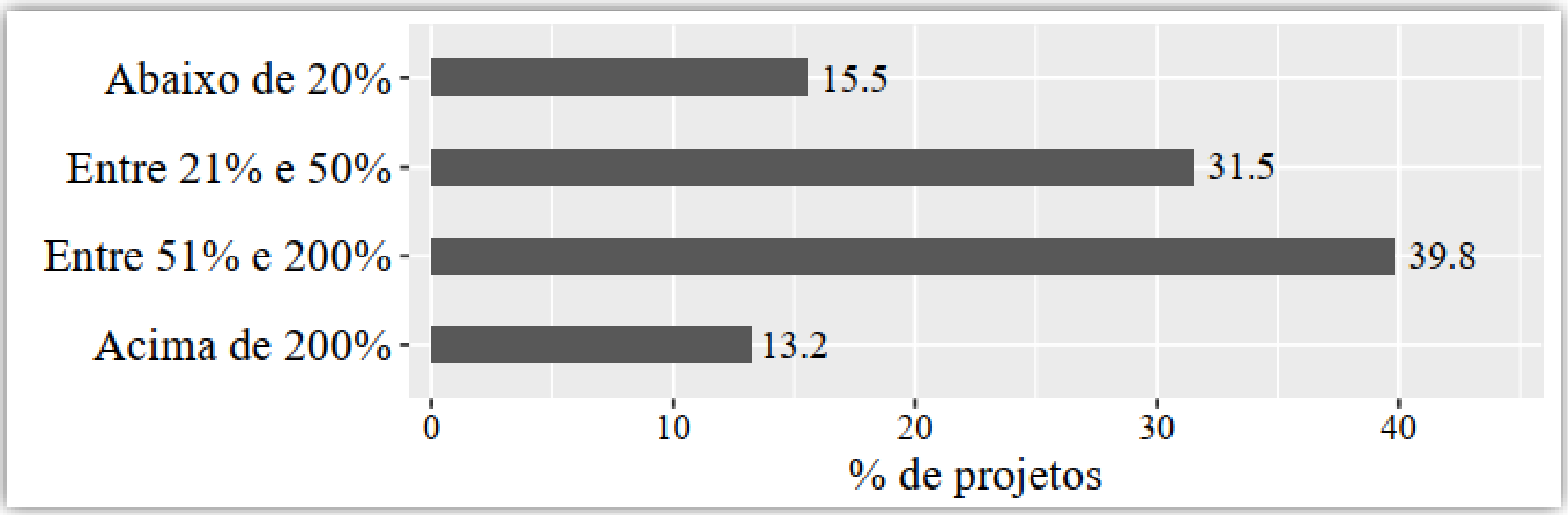
DESENVOLVIMENTO: MODELO WATHERFALL



CHAOS Report (1994) – Relatório produzido pela consultoria Standish Group: percentual de projetos que estourava seus **prazos** (para cada faixa de estouro). Mais de 55% dos projetos estourava os prazos planejados entre 51% e 200%.

Fonte: **VALENTE**, Marco Túlio. **Engenharia de Software Moderna**: Princípios e Práticas para Desenvolvimento de Software com Produtividade. 2020. Belo Horizonte.

DESENVOLVIMENTO: MODELO WATHERFALL



CHAOS Report (1994) – Relatório produzido pela consultoria Standish Group: percentual de projetos que estourava seus **orçamentos** (para cada faixa de estouro). Quase 40% dos projetos ultrapassava o orçamento entre 51% e 200%.

Fonte: **VALENTE**, Marco Túlio. **Engenharia de Software Moderna**: Princípios e Práticas para Desenvolvimento de Software com Produtividade. 2020. Belo Horizonte.

PUC Minas

PUC Minas

MÉTODOS ÁGEIS

MÉTODO ÁGIL

Manifesto para Desenvolvimento Ágil de Software

- **Indivíduos e interações** mais que processos e ferramentas
- **Software em funcionamento** mais que documentação abrangente
- **Colaboração com o cliente** mais que negociação de contratos
- **Responder a mudanças** mais que seguir um plano

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

Fonte: https://agilemanifesto.org/iso/ptbr/manifesto.html. O Manifesto Ágil é uma declaração de **12 princípios** e **4 valores** que ajudam a explorar melhores formas de processos de desenvolvimento de software.

MÉTODO ÁGIL

- O produto é dividido em várias **pequenas construções incrementais**, que são posteriormente utilizadas para um processo de interação
- Entrega o produto adequado ao cliente por meio **unidades pequenas e frequentes do projeto**
- Também referenciado como "**Processos Leves**"

MÉTODO ÁGIL



Iteração 1 (Sprint)

Semanas a 1 mês

Iteração 2 (Sprint)

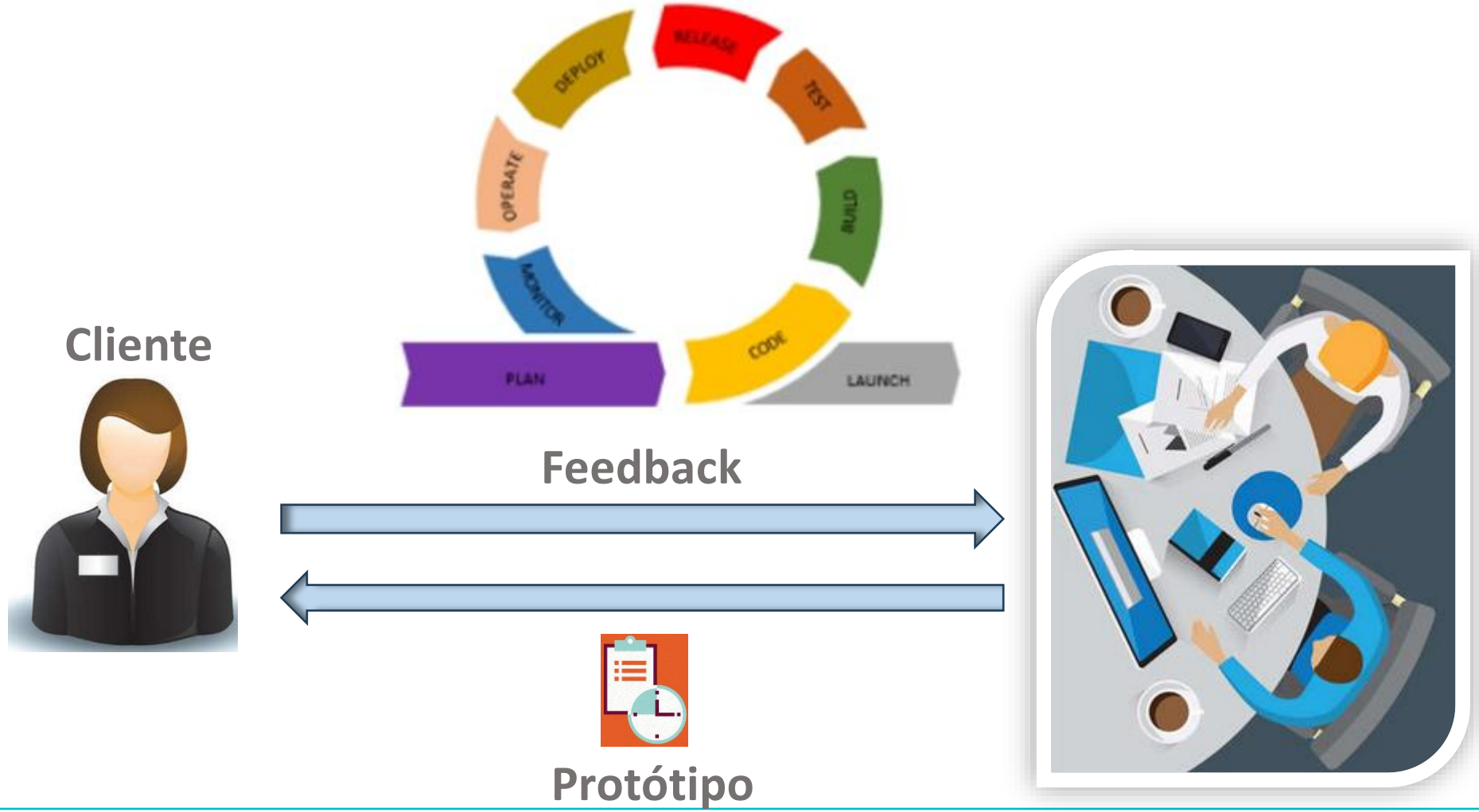
Semanas a 1 mês

Iteração... N (Sprint)

Semanas a 1 mês

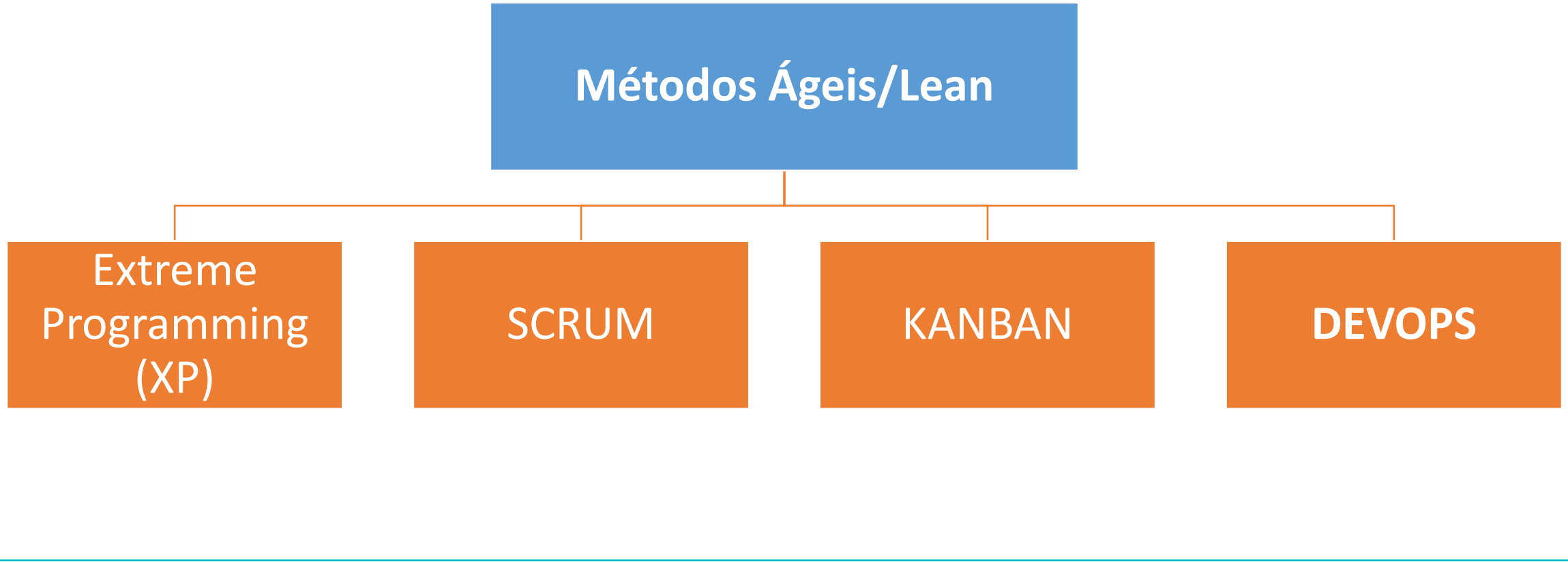
Fonte: Idealizado de EC-Council DevSecOps Engineer.

MÉTODO ÁGIL



Fonte: Idealizado de EC-Council DevSecOps Engineer.

MÉTODO ÁGIL


Métodos Ágeis/Lean
Extreme Programming (XP)
SCRUM
KANBAN
DEVOPS

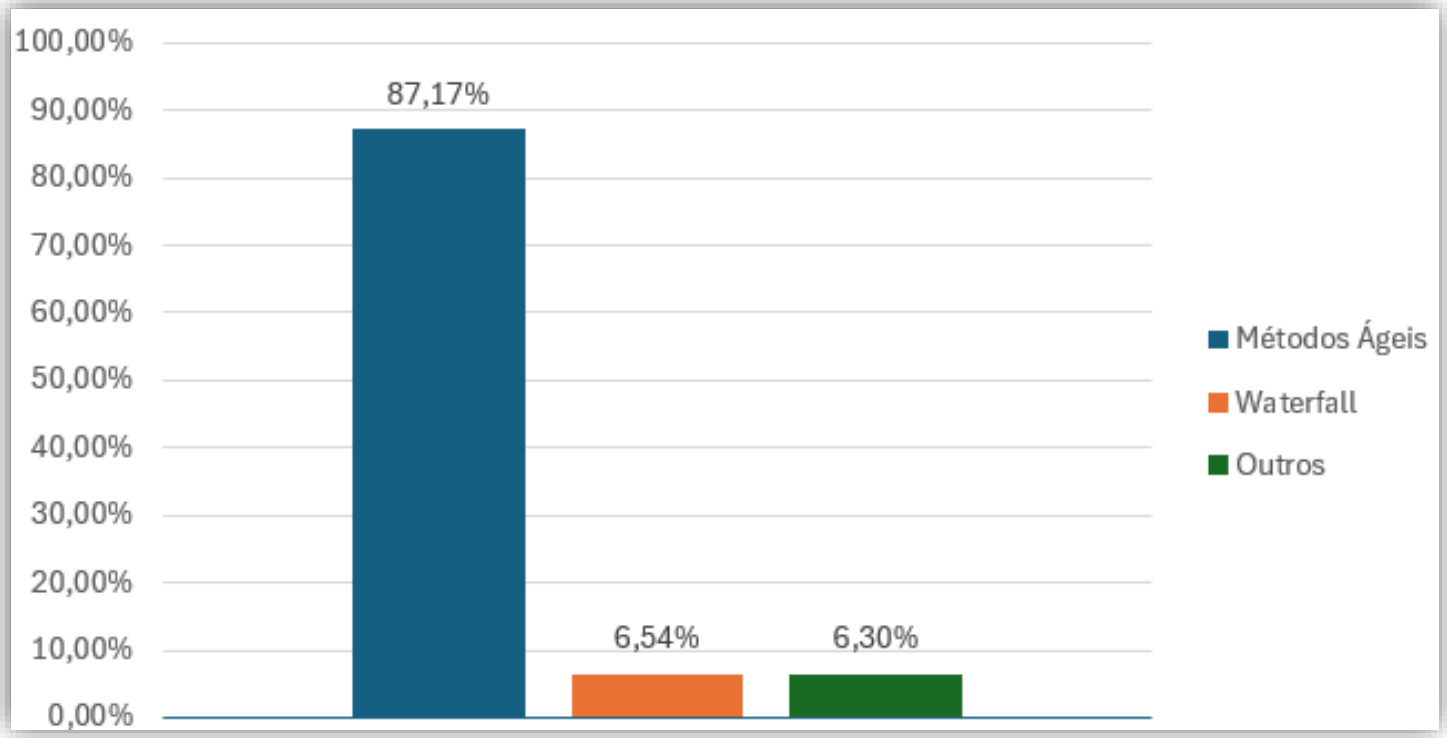
MÉTODO ÁGIL: CARACTERÍSTICAS

- **Extreme Programming (XP)**
  - Método ágil com foco exclusivo no **desenvolvimento de software**
  - **Inclui** conjunto de **práticas de programação** (programação pareada, desenvolvimento dirigido por testes (TDD), build automatizado, integração contínua)
- **SCRUM/KANBAN**
  - Método ágil para **gerenciamento de projetos** (de software, inclusive)
  - Não propõe práticas de programação

MÉTODO ÁGIL: CARACTERÍSTICAS

- Foco na **colaboração** e **comunicação** (relação cliente e desenvolvedores)
- Menor ênfase em planos detalhados (clareza plena dos requisitos nem sempre é possível, foco em feedback)
- Menor ênfase em **documentação** (somente o essencial)
- **Não há** uma **fase dedicada de design** (Big Design Up Front – BDUF), o **design é incremental** e **evolui com o sistema a cada iteração**
- Composição de equipes menores (uma dezena de desenvolvedores)
- Equipes de **Desenvolvimento** e **Operações** trabalham em **silos**

MÉTODO ÁGIL: ADOÇÃO



Em relação ao processo de desenvolvimento de software adotado pelas empresas e seus respondentes, destaca-se a adoção de **métodos ágeis** como **Scrum** (360 - **87,17%**). Além disso, ciclos de desenvolvimento prescritivos, como o ciclo **Cascata** e equivalentes representam **6,54%** (27) da amostra, enquanto **outros** tipos de processos representam **6,30%** (26).

Fonte: Surveying the Impacts of COVID-19 on the Perceived Productivity of Brazilian Software Developers. SBES 2020.

PUC Minas

PUC Minas

PROJETO E PRODUTO

PROJETO

- O projeto é caracterizado por ser um **empreendimento único** que **conduzido por pessoas** possa **atingir seus objetivos definidos**, respeitando os parâmetro de **custo**, **prazo** e **escopo**
- Possui **início**, **meio** e **fim** claramente definidos

PRODUTO

- O produto é orientado a valor, tendo o foco no cliente
- É um processo contínuo
- Em uma relação de produto e projeto, o projeto é um processo é o produto é o resultado deste processo
- O sucesso de um projeto não implica necessariamente no sucesso de um produto

PROJETO E PRODUTO: MINDSET



Fonte: MUNIZ, Antônio. Jornada Ágil do Produto. 2020. Brasport. **Mindset de Projeto**.

PROJETO E PRODUTO: MINDSET



Fonte: MUNIZ, Antônio. Jornada Ágil do Produto. 2020. Brasport. **Mindset de Produto**.

MINDSET DO PRODUTO

O problema não é que nossas organizações percebam que precisam se transformar; o problema é que as organizações estão usando **estruturas gerenciais** e **modelos de infraestrutura anteriores** para **gerenciar seus negócios nesta (Mike Kersten, 2018)**

Fonte: MUNIZ, Antônio. Jornada Ágil do Produto. 2020. Brasport. **Mindset de Produto**.

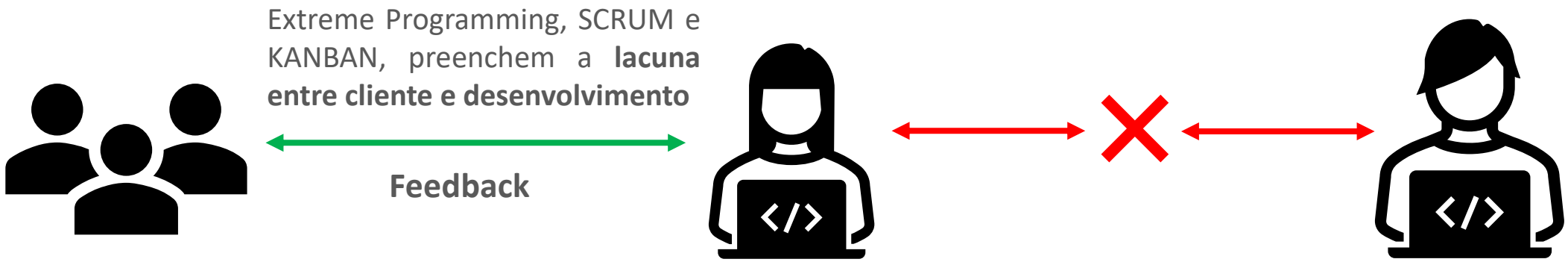
PUC Minas

PUC Minas

DESENVOLVIMENTO (DEV) E OPERAÇÕES (OPS)

DEVOPS

DEVOPS


Extreme Programming, SCRUM e KANBAN, preenchem a **lacuna entre cliente e desenvolvimento**
Feedback
Cliente e Requisitos
Desenvolvimento e Teste
Operações e Infraestrutura

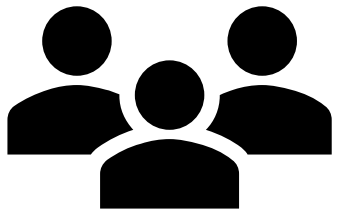
DEVOPS

DEV

- **Aumentar** o valor para o negócio
- **Agilidade** para inovar
- Requisitos funcionais

OPS

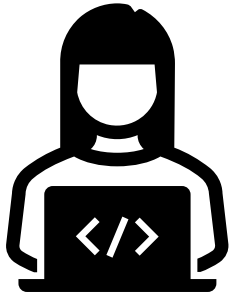
- **Proteger** valor para o negócio
- **Estabilidade** do ambiente
- Requisitos não funcionais



Cliente e Requisitos

Extreme Programming, SCRUM e KANBAN, preenchem a **lacuna entre Cliente e Desenvolvimento**

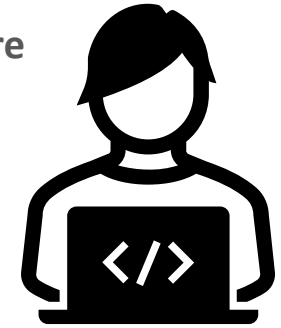
Feedback



Desenvolvimento e Teste

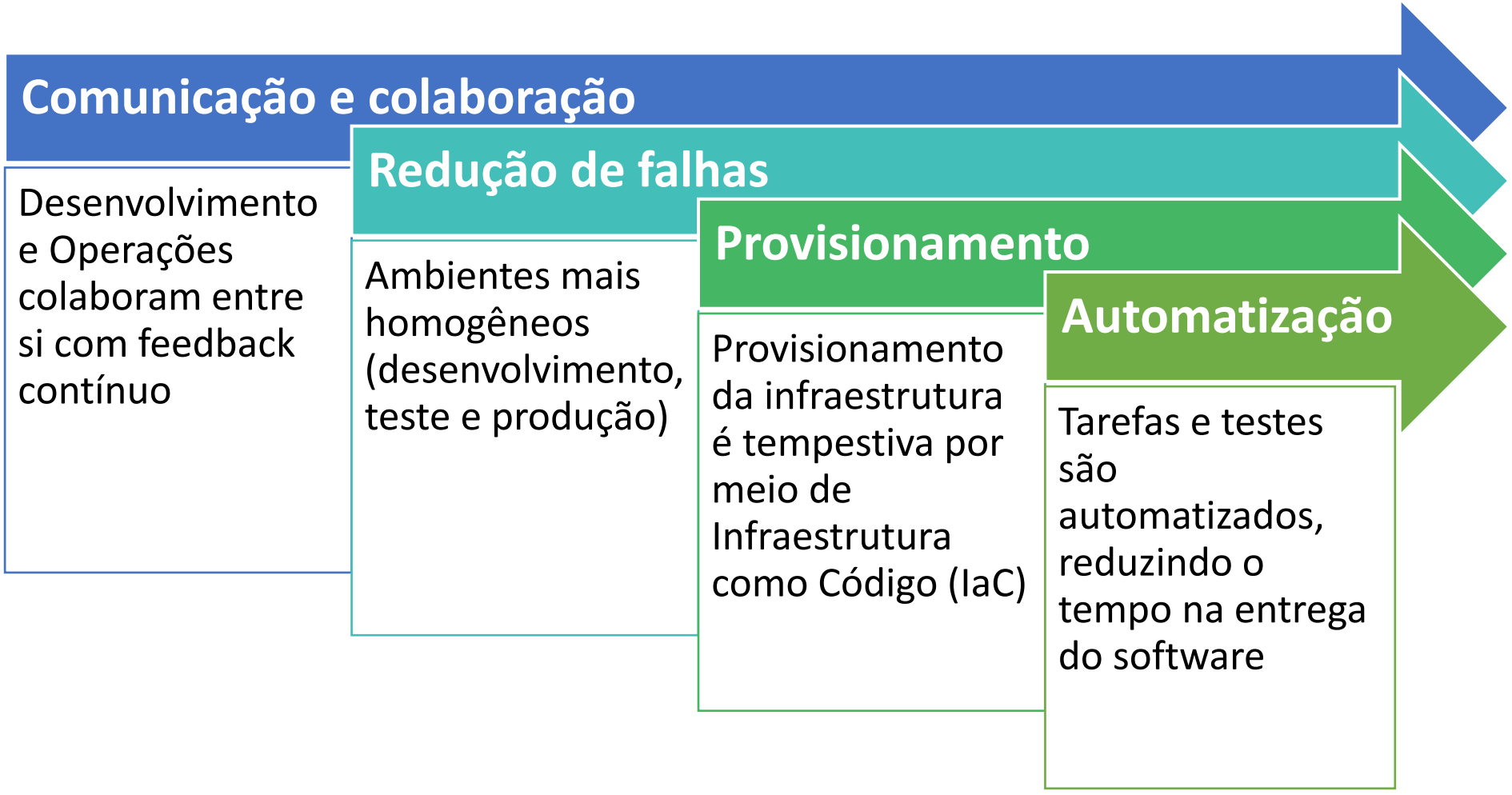
DevOps preenche a **lacuna entre Desenvolvimento e Operações**

Feedback

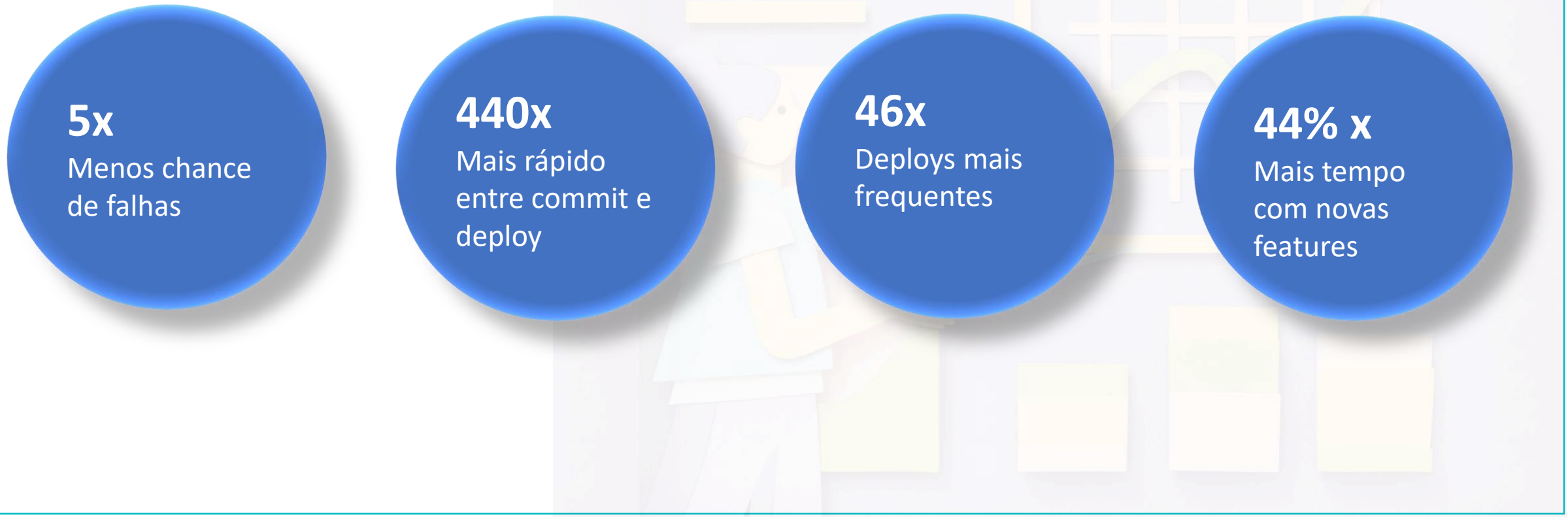


Operações e Infraestrutura

FATORES DE SUCESSO DO DEVOPS

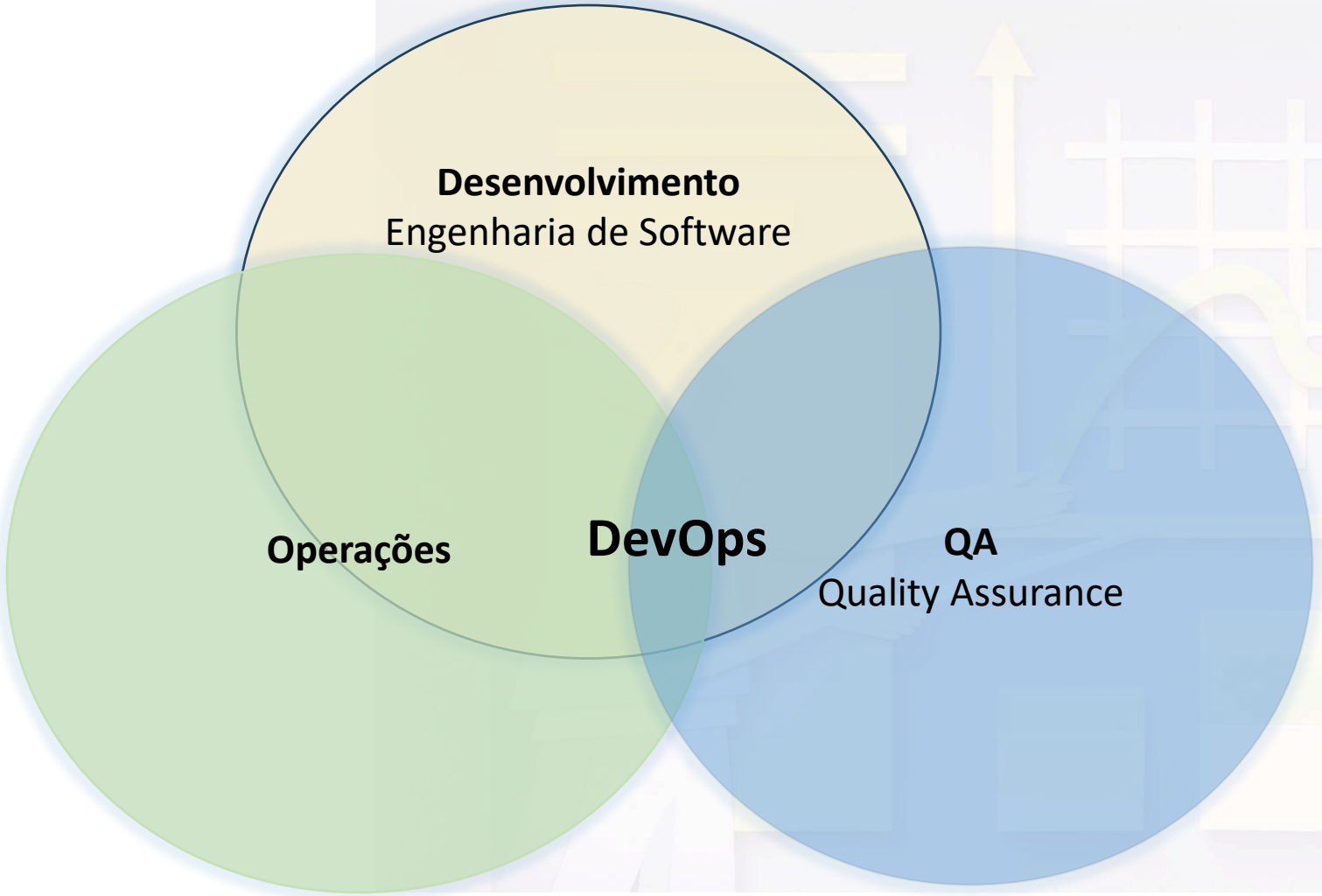


FATORES DE SUCESSO DO DEVOPS



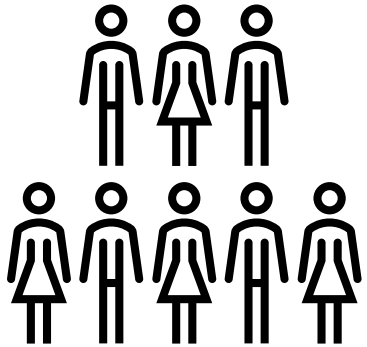
Fonte: MUNIZ, Antônio. Jornada DevOps. 2020. Brasport. **Benefícios com a adoção de DevOps**. Pesquisa com várias empresas e segmentos de mercado no mundo.

DEVOPS


Desenvolvimento
Engenharia de Software
Operações
DevOps
QA
Quality Assurance

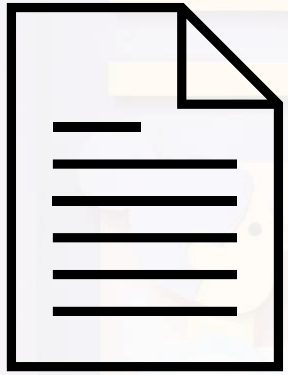
BUILDING BLOCKS DEVOPS

Pessoas



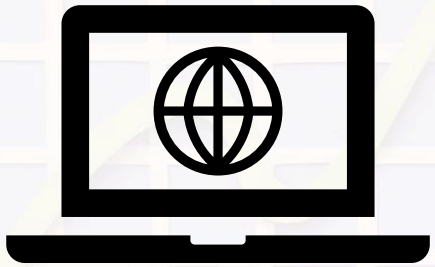
Todas as equipes devem colaborar e trabalharem juntas

Processos



Eficiência de processos ajuda a organização a melhorar a produtividade

Produtos/Tecnologia



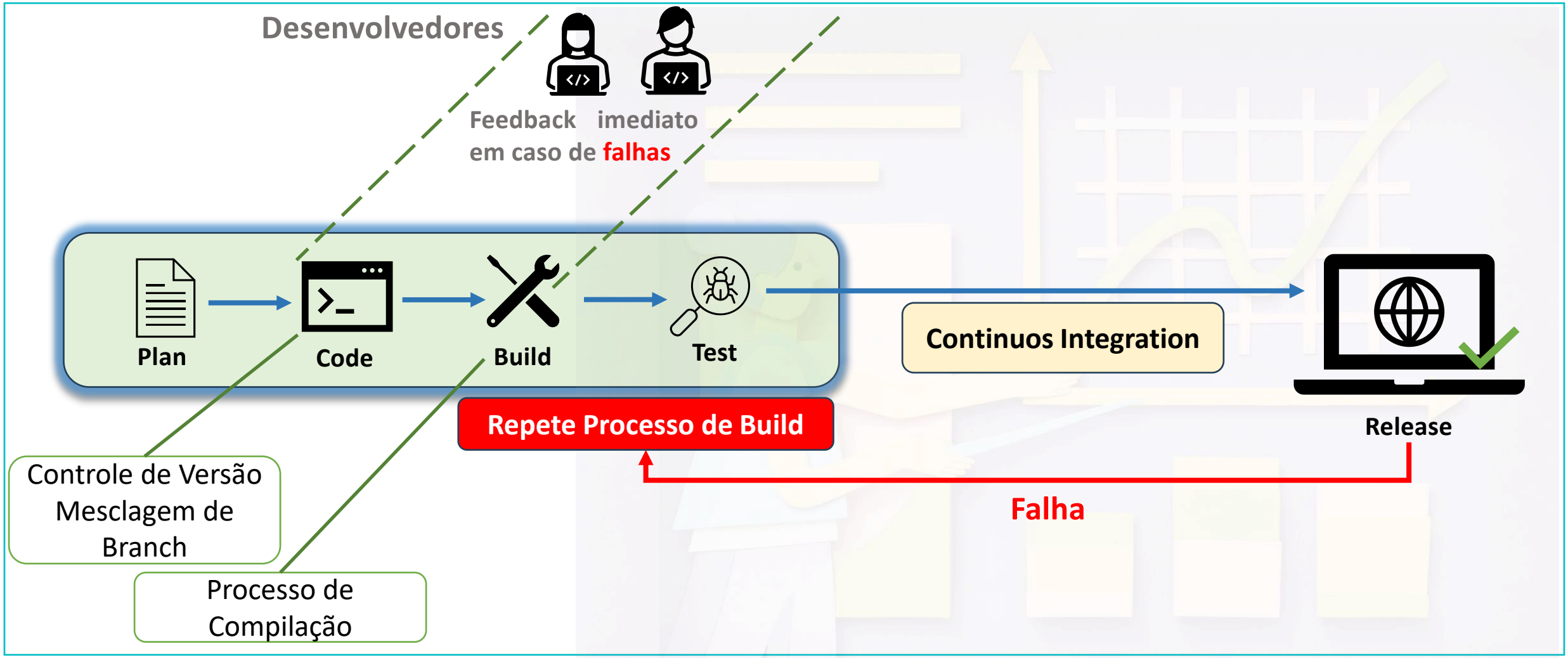
Minimizar débito técnico e aumentar a agilidade

Fonte: Idealizado de EC-Council DevSecOps Engineer.

CONTINUOS INTEGRATION (CI)

- **Integração Contínua** (CI) (XP Programming) **testa** e **mescla** automaticamente as alterações para ajudar a manter a integridade da base de código
- Auxilia na detecção e correção de bugs nos estágios iniciais, executando testes automatizados
- Fornece feedback imediato sobre os problemas à medida que eles ocorrem
- Reduz riscos ao tornar o processo de implantação fácil e previsível

CONTINUOS INTEGRATION (CI)

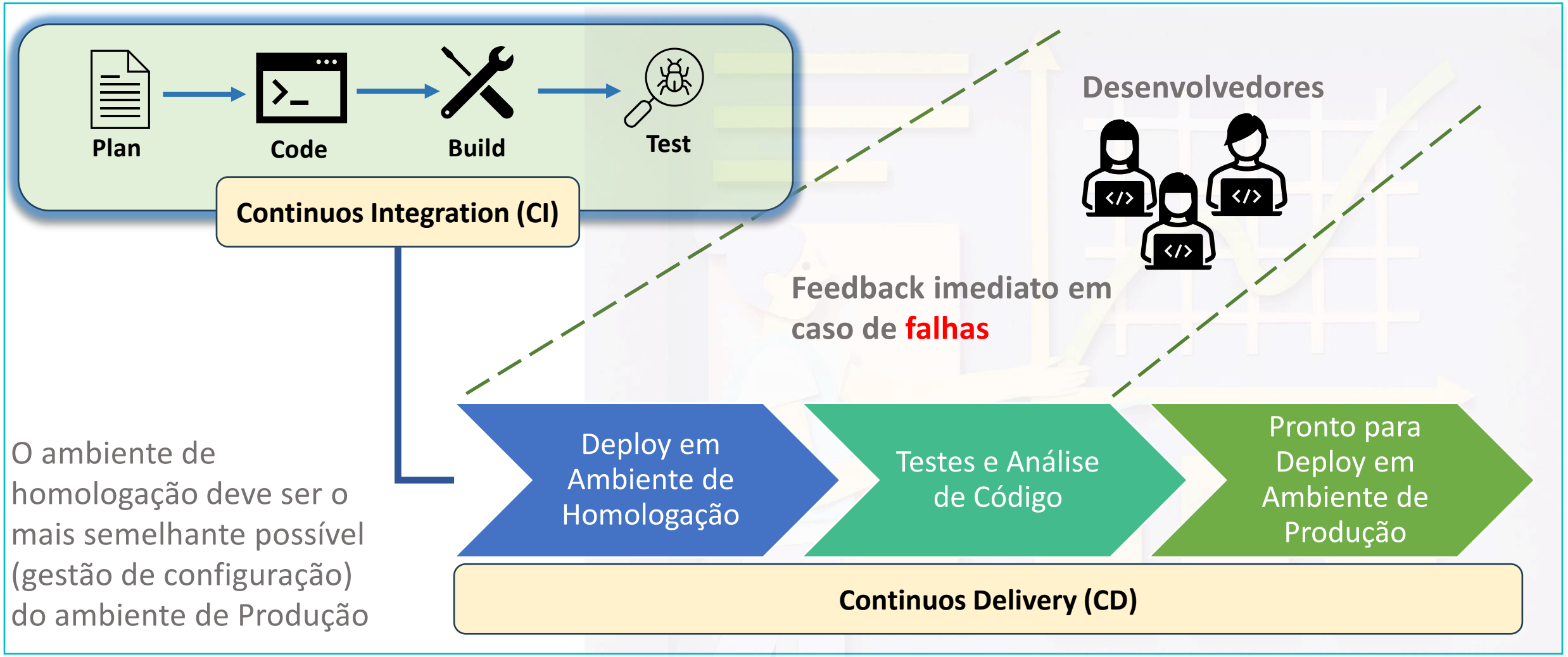


Fonte: Idealizado de EC-Council DevSecOps Engineer.

CONTINUOS DELIVERY (CD)

- A Entrega Contínua (CD) é uma **extensão do CI**
- Usa ferramentas avançadas para testar e **validar completamente uma compilação** e a passa para a implantação em **ambiente de homologação**
- **Elimina a complexidade de implantação de software**
- Permite **lançamentos (releases) com mais frequência** e **acelera o ciclo de feedback** com os clientes
- Reduz a pressão para tomar decisões sobre pequenas mudanças, incentivando **iterações mais rápidas**

CONTINUOS DELIVERY (CD)

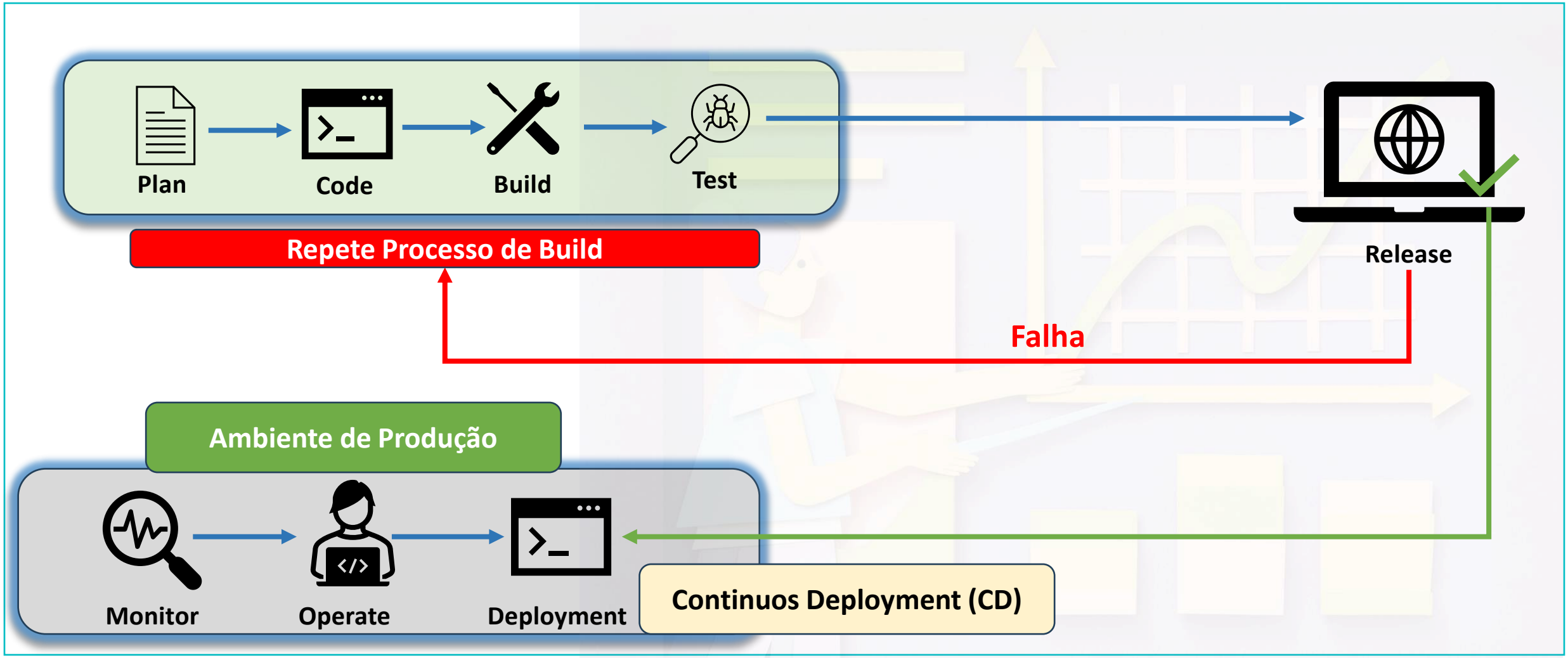


Fonte: Idealizado de EC-Council DevSecOps Engineer.

CONTINUOS DEPLOYMENT (CD)

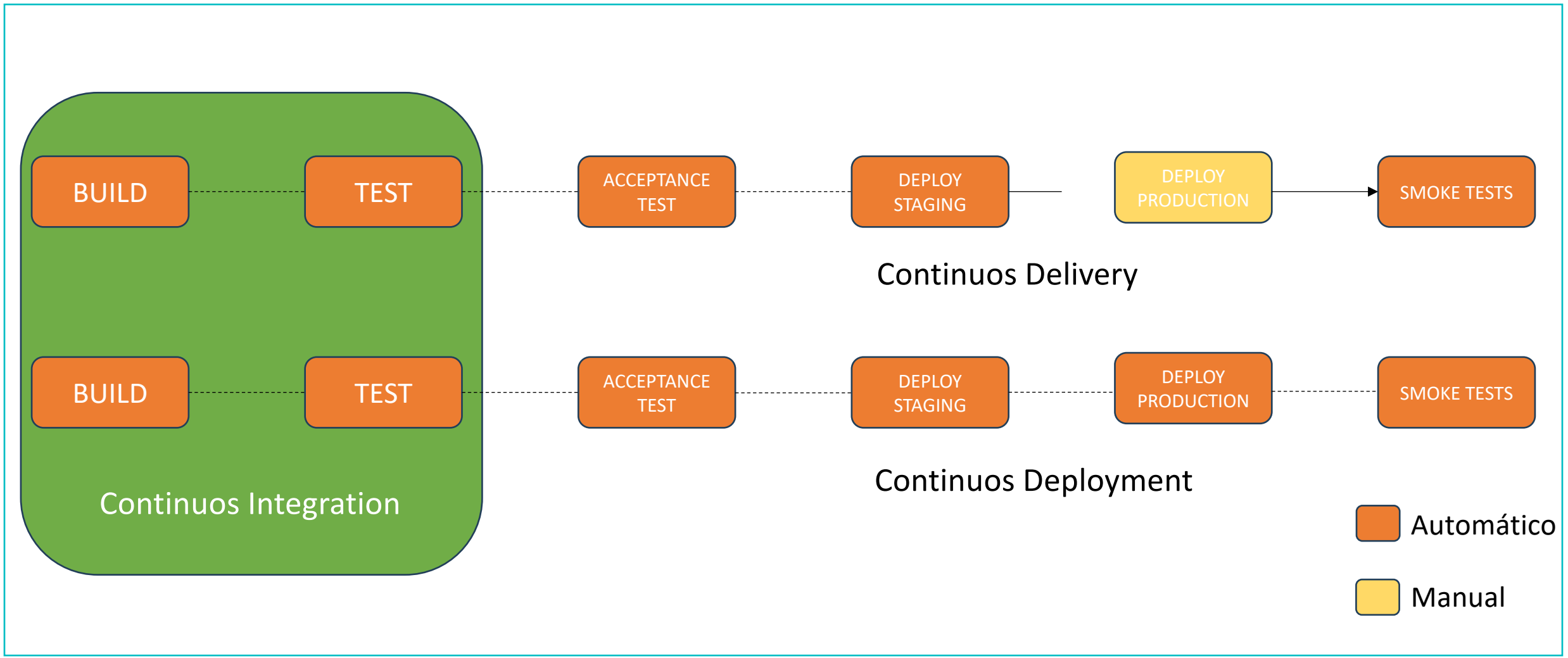
- Atua na implantação contínua de alterações de configuração, novos recursos e correções erros para o ambiente de produção
- Automatiza as tarefas repetitivas para o processo de implantação de software
- Garante implantação sem comprometer a segurança e minimiza erros
- Permite que as organizações integrem processos e equipes em um único pipeline unificado

CONTINUOS DEPLOYMENT (CD)



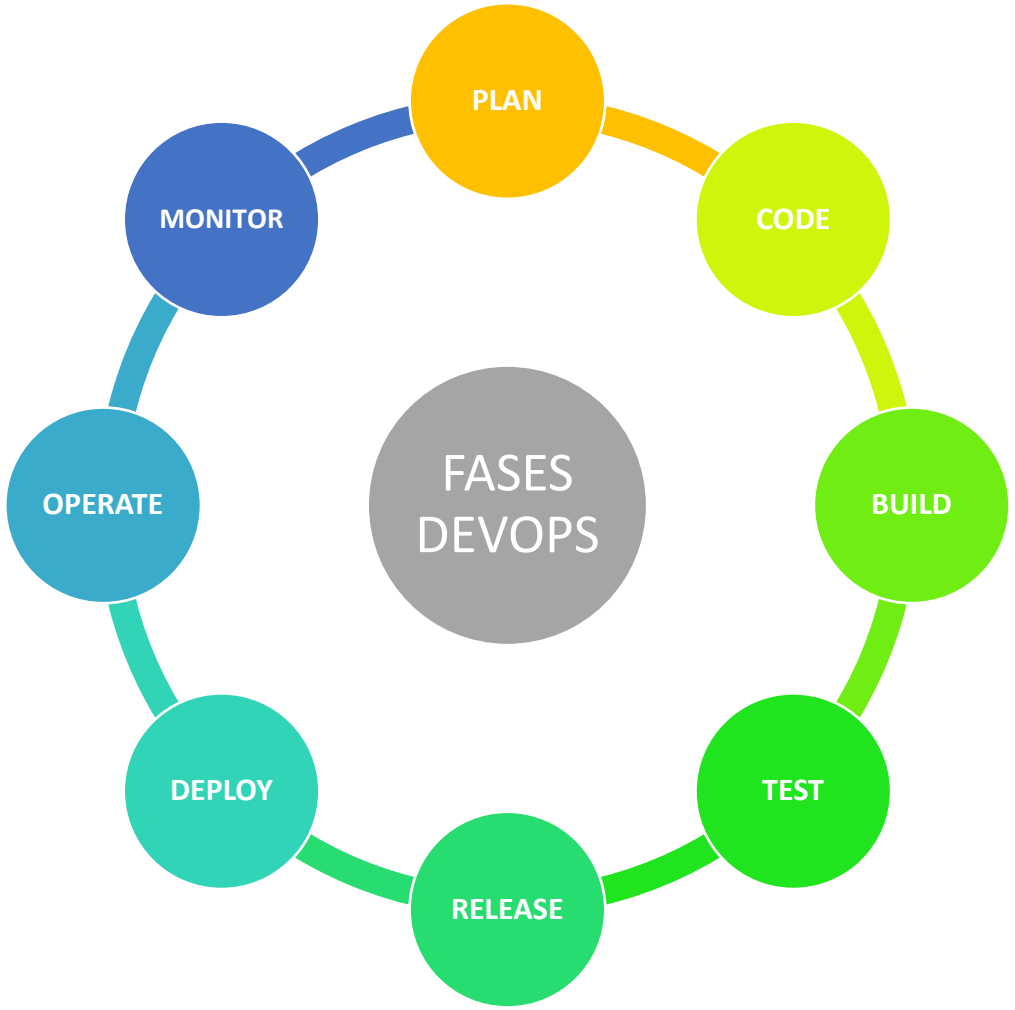
Fonte: Idealizado de EC-Council DevSecOps Engineer.

VISÃO GERAL DO PIPELINE CI/CD



Fonte: Idealizado de EC-Council DevSecOps Engineer.

FASES TÍPICAS DO MÉTODO DEVOPS



- **PLAN**: requisitos das partes interessadas e clientes, e feedback, sprints
- **CODE**: escrita de código, kit de desenvolvimento, plugins, linguagens
- **BUILD**: commit, pull request, aprovação, merge, artefatos de software
- **TEST**: UAT, teste de segurança, teste de carga, integração
- **RELEASE**: processo de build é planejado, agendado e controlado em diferentes ambientes
- **DEPLOY**: todo o produto de software e arquivos correlatos são implantados no ambiente de produção (máquinas virtuais, contêineres)
- **OPERATE**: o produto de software é entregue para o cliente em pleno funcionamento pela equipe de Operações
- **MONITOR**: monitoramento contínuo do software, métricas, desempenho

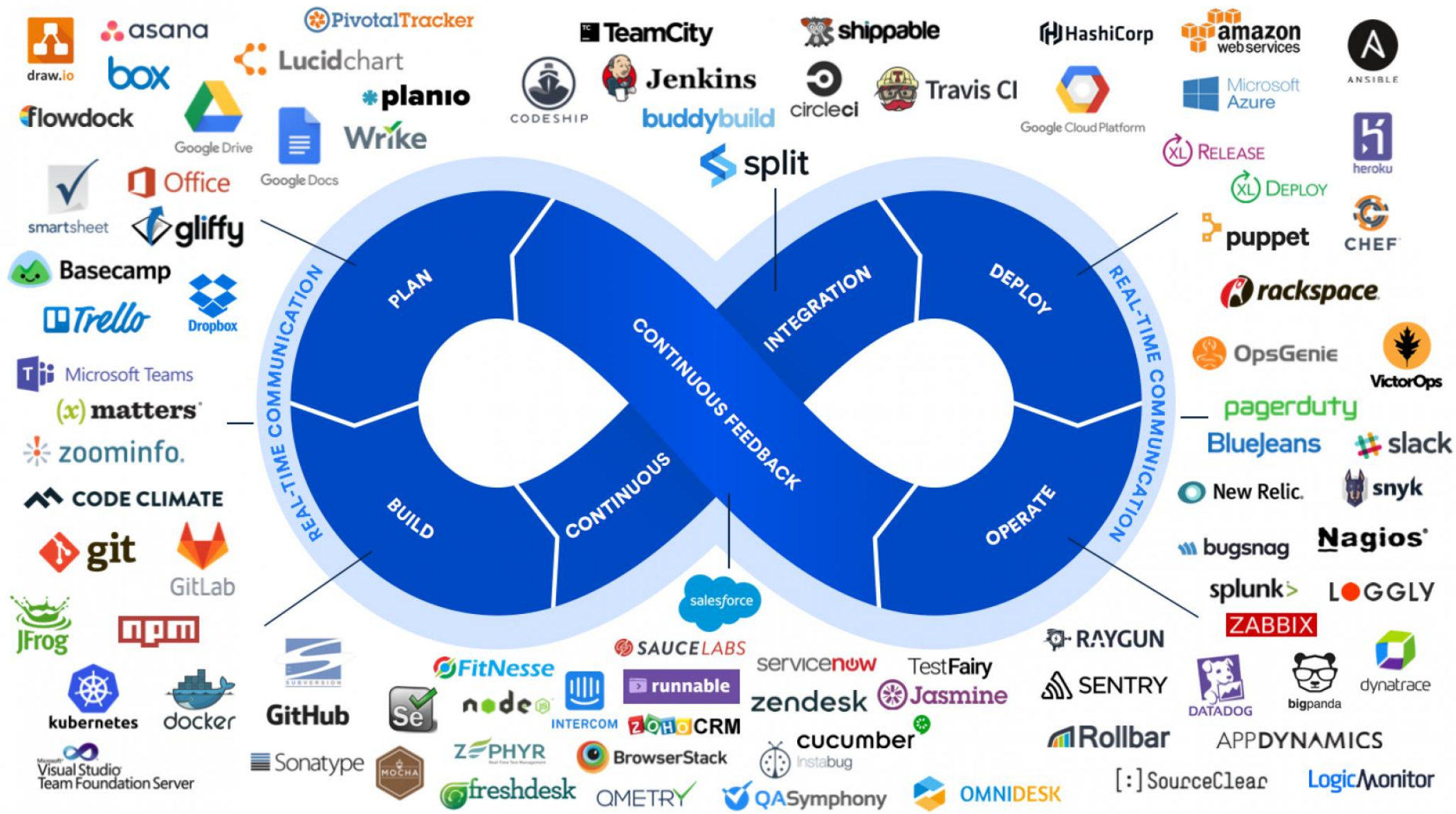
PRÁTICAS GERAIS NO MÉTODO DEVOPS

- Infrastructure as a Code (IaC)
- Continuos Integration
- Testes Automatizados
- Continuos Deployment
- Gerenciamento de Release
- Monitoramento de Desempenho de Aplicações
- Escalonamento Vertical, Horizontal e Diagonal

- Monitoramento da Disponibilidade
- Gerenciamento de Configuração
- Feature Toggle Flags
- Provisionamento e Desprovisionamento Automatizado
- Self-Service Envirolment
- Rollback automatizado
- Suporte ao Desenvolvimento Baseado em Hipótese

Fonte: Idealizado de EC-Council DevSecOps Engineer.

DEVOPS TOOLCHAIN



Fonte: https://www.zippyops.com/how-to-build-a-robust-devops-toolchain

PUC Minas

PUC Minas

SEGURANÇA

OBJETIVO

Compreender os principais conceitos primários e fundamentais que envolvem a segurança da informação no **contexto de DevSecOps**

PROPRIEDADES DA INFORMAÇÃO

CID

- Confidencialidade
- Integridade
- Disponibilidade

Parkerian Hexad (Donn Parker, 1998)

- Posse/Controle
- Autenticidade (FIPS-99)
- Utilidade

Cube Model (McCumber, 2004)

- Dados em trânsito
- Dados em repouso

CIANA+PS

- Confidencialidade
- Integridade
- Disponibilidade
- Irretratabilidade
- Autenticidade
- Privacidade
- Safety

RISCO

- **Risco** é o **efeito da incerteza sobre os objetivos** (ISO 31000)
- Uma **medida** da extensão em que uma entidade é ameaçada por uma circunstância ou evento potencial e normalmente uma função de: **(1)** os **impactos** adversos que surgiriam se a circunstância ou evento ocorrer; e **(2)** a **probabilidade** de ocorrência (NIST)

RISCOS DE SEGURANÇA DA INFORMAÇÃO

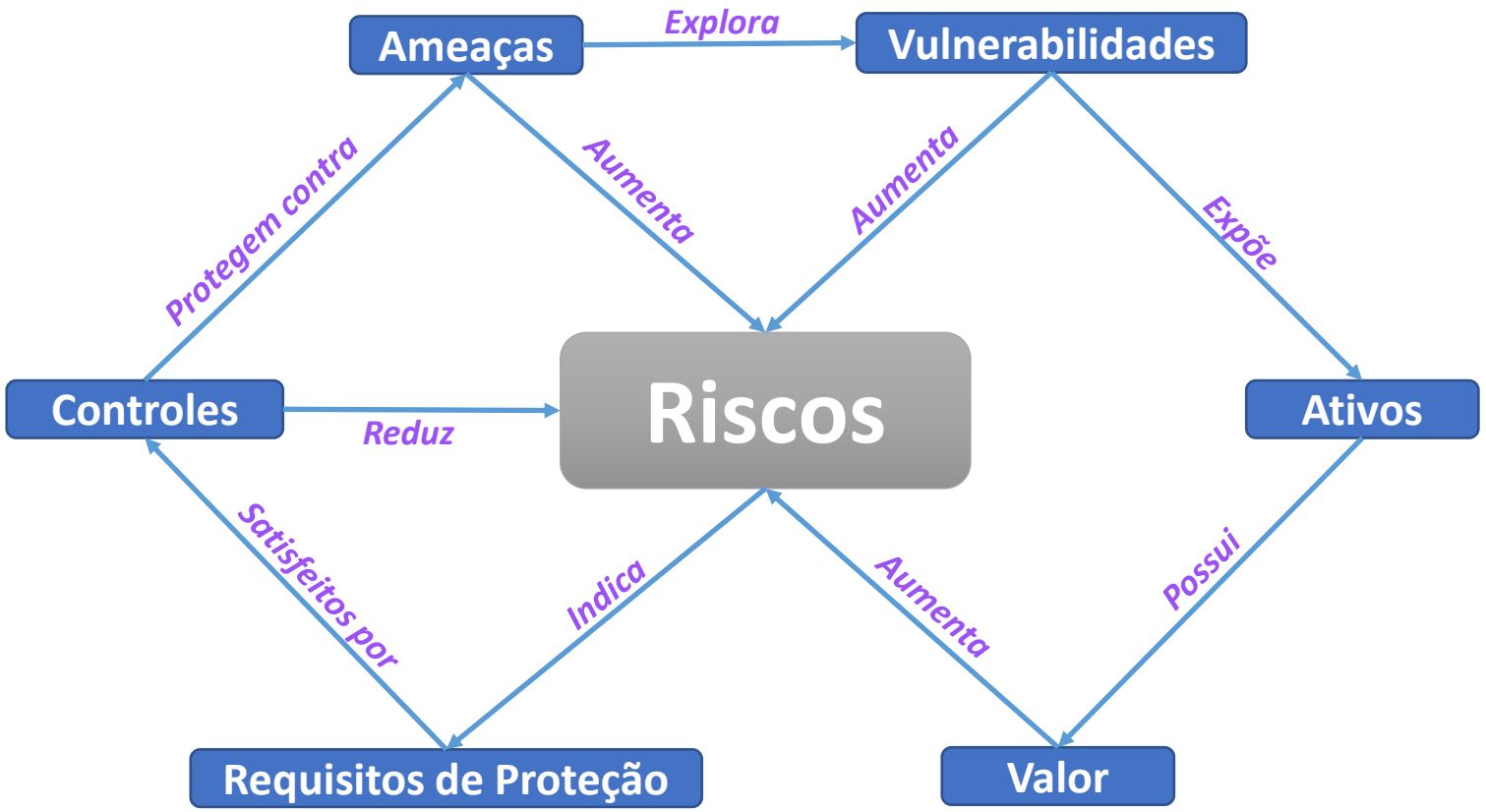
- São aqueles que surgem através da **perda da confidencialidade**, **integridade** ou **disponibilidade** de informações ou **sistemas de informação**, e considerar os **impactos na organização** (incluindo ativos, missão, funções, imagem ou reputação), indivíduos, outras organizações e a Nação. (NIST)

RISCO

- Probabilidade
- Impacto
- Ameaça
- Vulnerabilidade



RELAÇÃO DOS CONCEITOS



SECURITY CONTENT AUTOMATION PROTOCOL (SCAP)

CPE	• **Common Platform Enumeration:** usa uma sintaxe semelhante aos Uniform Resource Identifiers (URI). CPE é um formato de **nomenclatura padronizado** usado para **identificar sistemas** e **software**
CVE	• **Common Vulnerabilities and Exposures:** uma lista de registros onde cada item contém um **identificador exclusivo** usado para **descrever vulnerabilidades publicamente conhecidas**. Identificadores exclusivos começam com CVE, seguido pelo ano de identificação e um número exclusivo - **CVE-YEAR-#####**
CCE	• **Common Configuration Enumeration:** semelhante à CVE, exceto pelo foco em **problemas de configuração** que podem resultar em uma vulnerabilidade
CVSS	• **Common Vulnerability Scoring System:** representa uma pontuação numérica para refletir a **gravidade de uma vulnerabilidade**. A pontuação varia de 0 a 10 com classificações qualitativas

CVSS 3.1: BASE SCORE DA GRAVIDADE TÉCNICA

SEVERIDADE	BASE SCORE
Crítica	9.0 – 10.0
Alta	7.0 – 8.9
Média	4.0 – 6.9
Baixa	0.1 – 3.9
Nenhuma	0.0

EXPLOIT PREDICTION SCORING SYSTEM (EPSS)

EPSS
Exploit Prediction Scoring System

Mission

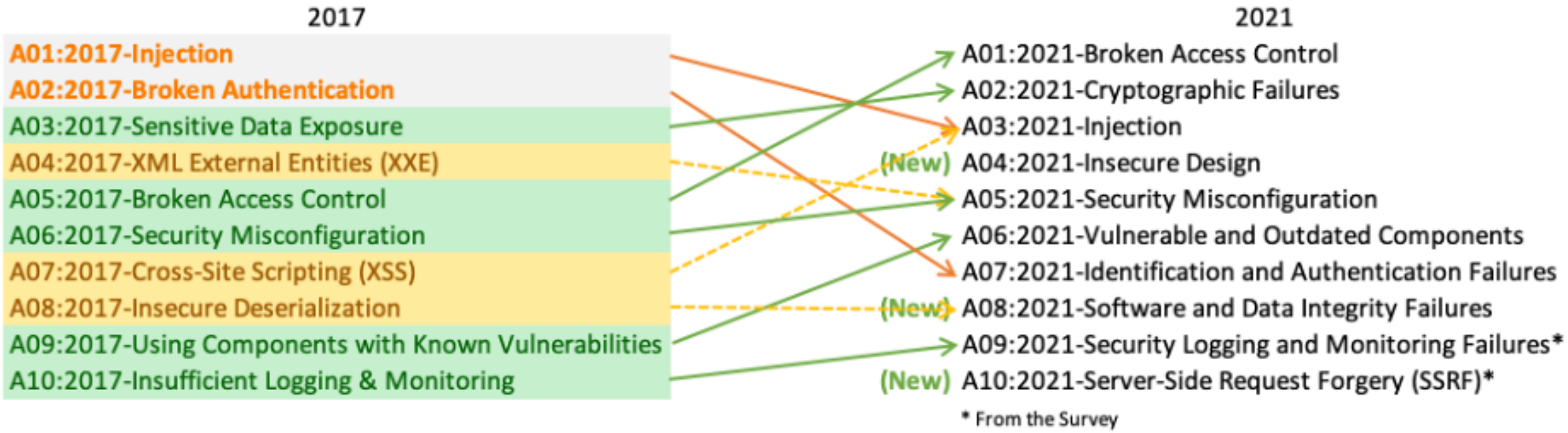
The Exploit Prediction Scoring System (EPSS) is a data-driven effort for estimating the likelihood (probability) that a software vulnerability will be exploited in the wild. Our goal is to assist network defenders to better prioritize vulnerability remediation efforts. While other industry standards have been useful for capturing innate characteristics of a vulnerability and provide measures of severity, they are limited in their ability to assess threat. EPSS fills that gap because it uses current threat information from CVE and real-world exploit data. The EPSS model produces a probability score between 0 and 1 (0 and 100%). The higher the score, the greater the probability that a vulnerability will be exploited.

Top: 5 | Life Cycle: Code, Build, Deploy, Run

CVE	CVSS	Severity	EPSS	Risk Factors
CVE-2022-22965	9.8	Critical	97%	Critical severity, Exploit exists -..., +7
CVE-2017-8046	9.8	Critical	49%	Critical severity, Exploit exists -..., +4
CVE-2022-1471	9.8	Critical	1%	Critical severity, Exploit exists -..., +6
CVE-2019-5413	9.8	Critical	1%	Critical severity, Exploit exists -..., +5
CVE-2021-3918	9.8	Critical	0%	Critical severity, High severity, +5

Referência: https://www.first.org/epss/ | https://www.paloaltonetworks.com/blog/prisma-cloud/epss-scores/

OWASP TOP 10

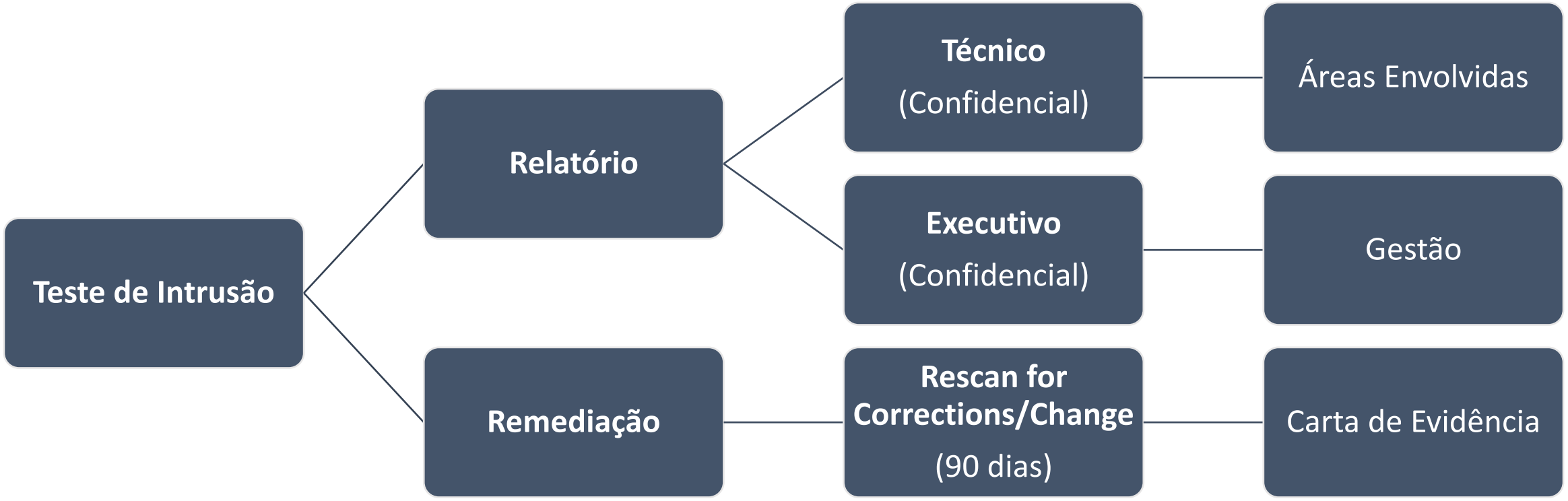


Referência: https://owasp.org/Top10/. Em novembro de 2025, será anunciado o OWASP Top 10 2025.

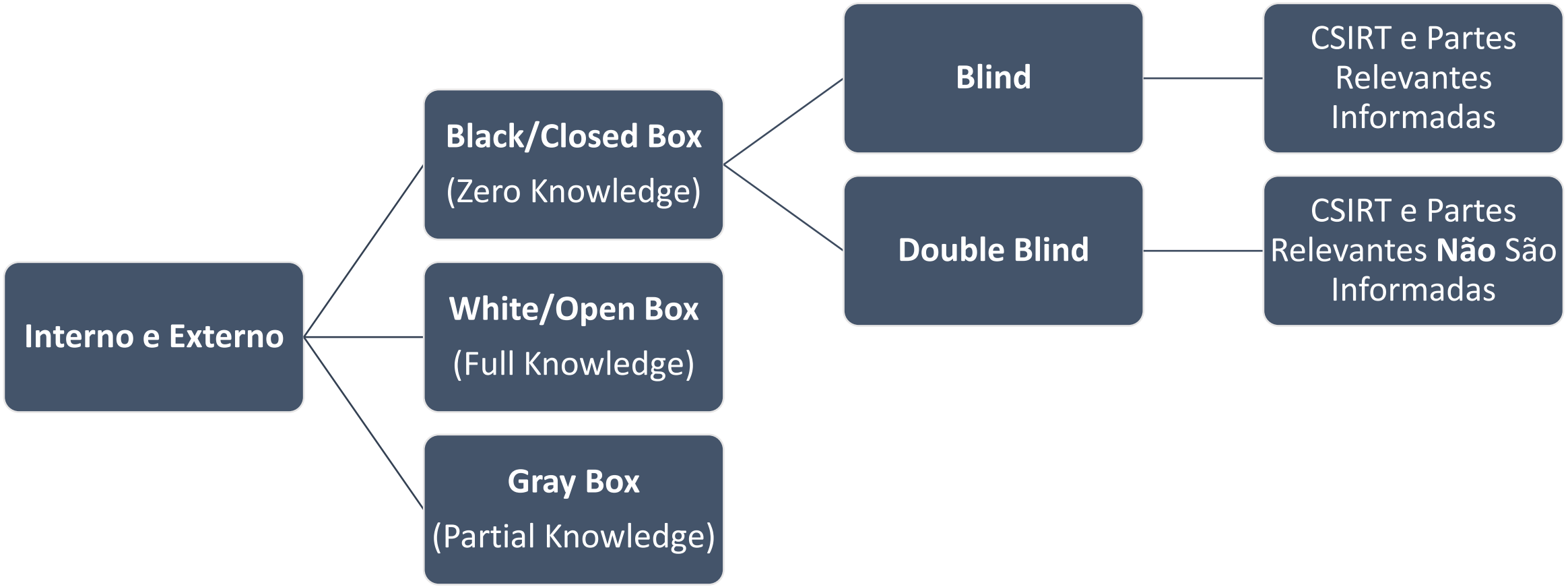
PENTEST | TESTE DE INTRUSÃO

- Metodologia de teste em que os testadores visam componentes computacionais individuais ou o aplicativo como um todo para determinar se **vulnerabilidades podem ser exploradas** (**vulnerabilidade desconhecidas**) para comprometer o aplicativo, dados ou recursos de ambiente

PENTEST DE TERCEIRA PARTE



MÉTODOS DE AVALIAÇÃO



Então... Por que DevSecOps e não SecDevOps?

DEVSECOPS

DEV

- **Aumentar** o valor para o negócio
- **Agilidade** para inovar
- Requisitos funcionais

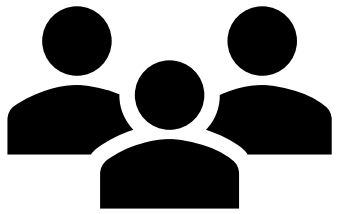
SEC

- **Segurança integrada** e **automatizada** no **Pipeline**
- **Mitigação de riscos de segurança** no SDLC e do produto

OPS

- **Proteger** valor para o negócio
- **Estabilidade** do ambiente
- Requisitos não funcionais

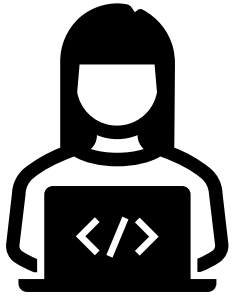
Extreme Programming, SCRUM e KANBAN, preenchem a **lacuna entre Cliente e Desenvolvimento**



Cliente e Requisitos

Feedback



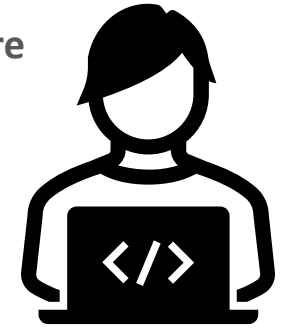


Desenvolvimento e Teste

DevOps preenche a **lacuna entre Desenvolvimento e Operações**

Feedback

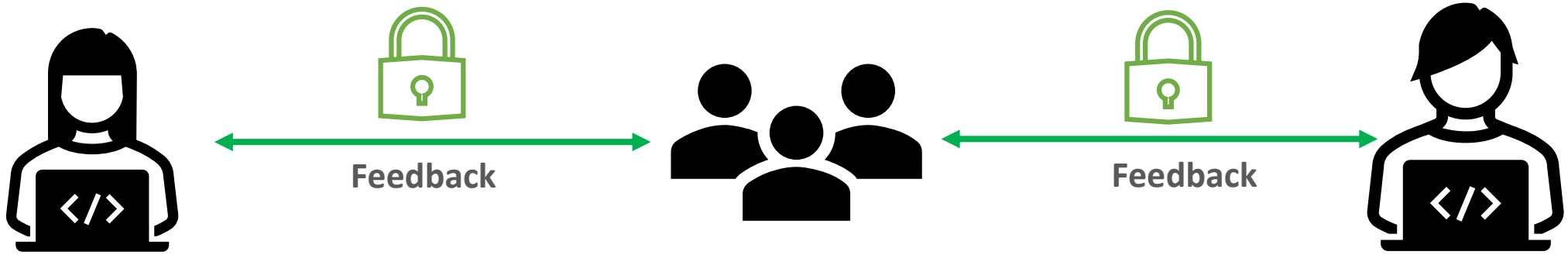




Operações e Infraestrutura

DEVSECOPS

Segurança preenche a **lacuna entre Desenvolvimento e Operações**



Desenvolvimento e Teste

Segurança da Informação

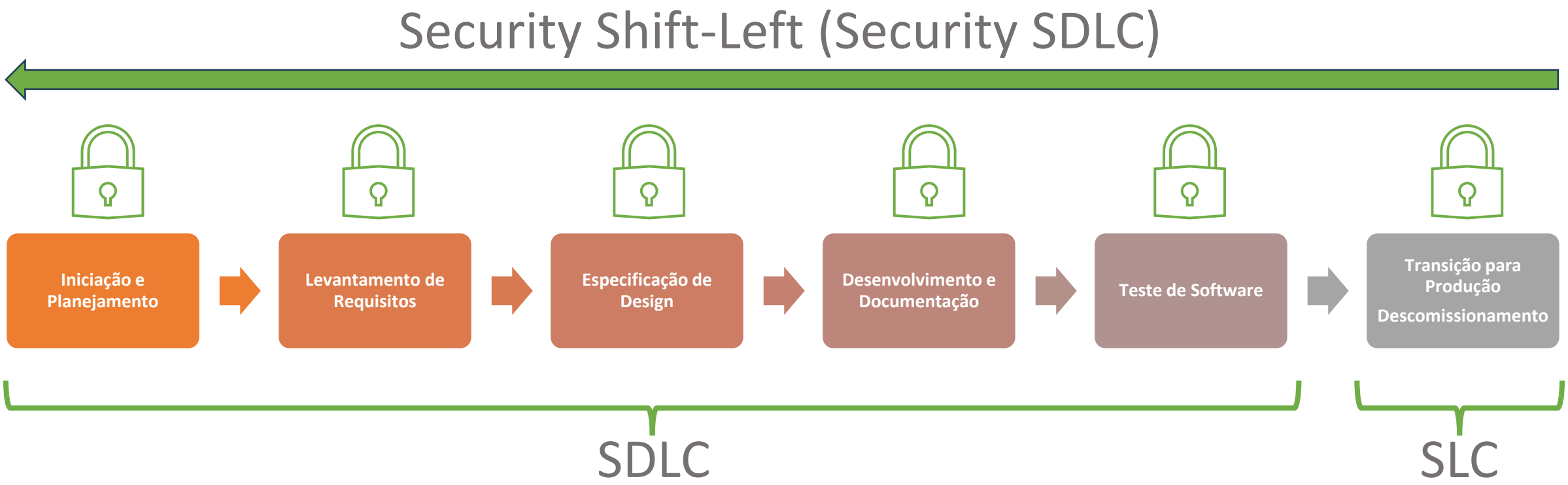
Operações e Infraestrutura

Dev

Sec

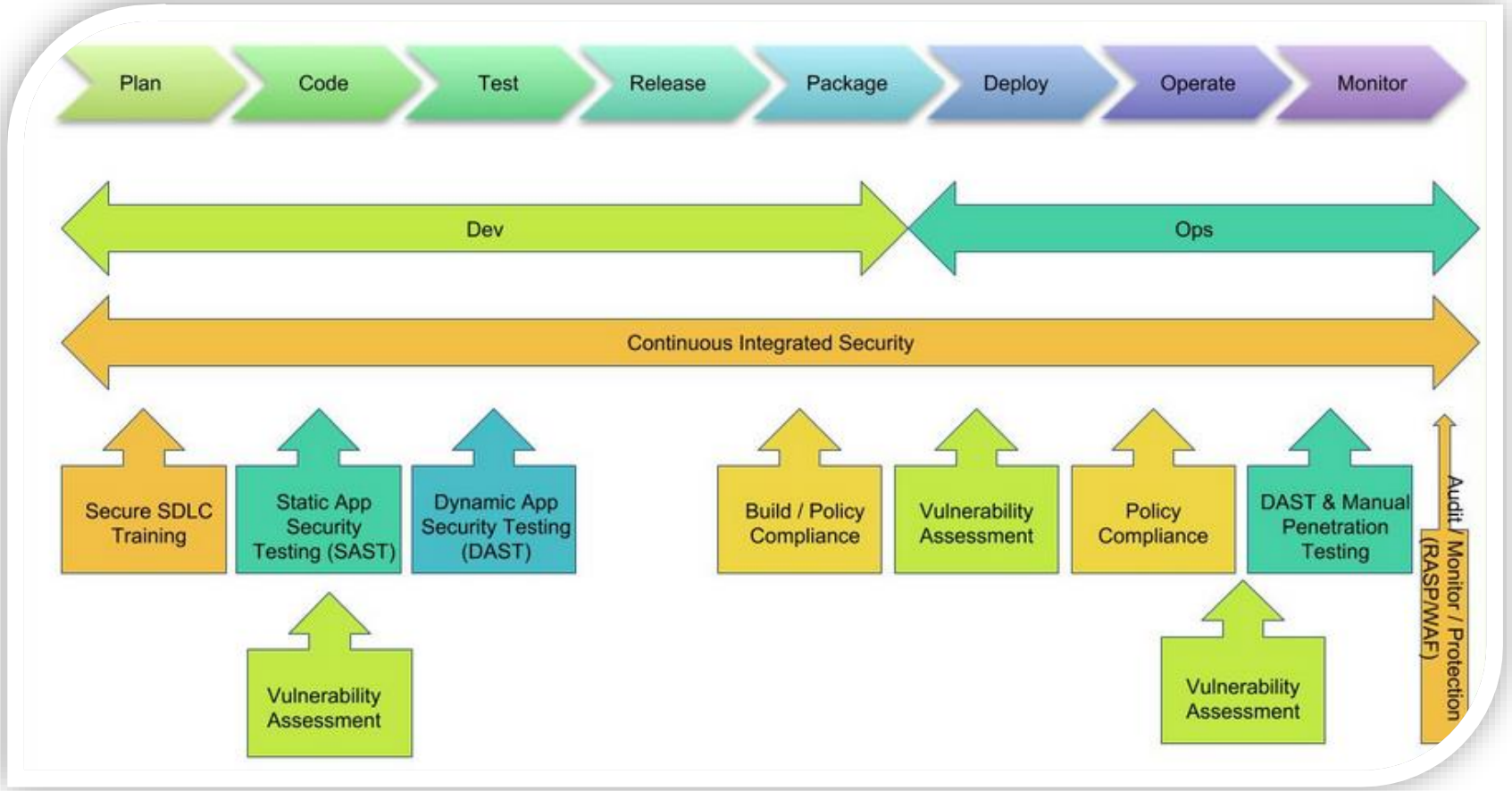
Ops

FASES TÍPICAS DO SDLC

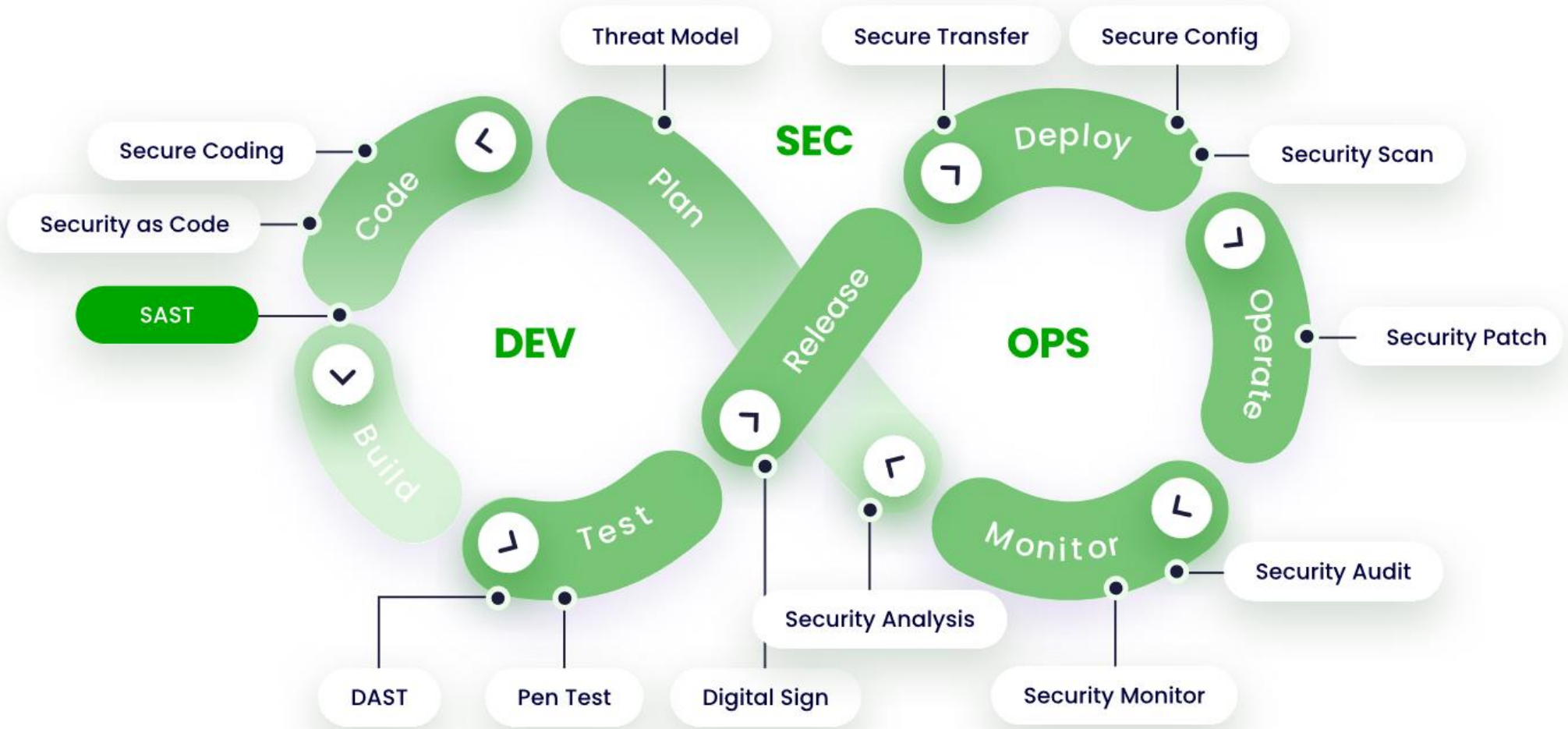


SHIFT-LEFT SECURITY

Na abordagem **Shift-left**, tem-se como premissa que a segurança deve ser sempre adotada desde o início do projeto



Fonte: https://www.e-spincorp.com/from-devops-shift-left-testing-to-devsecops-shift-left-security/



Fonte: https://www.mend.io/blog/sast-static-application-security-testing/

PUC Minas

PUC Minas

DESAFIO I

DESAFIO

Uma ferramenta que analisa vulnerabilidades em bibliotecas incorporadas ao software detectou um conjunto de vulnerabilidades. A área de segurança sumarizou as vulnerabilidades como se segue:

ID	AMBIENTE	CVSS	EPSS	PRIORIZAÇÃO
1	Frontend	Média	97%	
2	Backend	Crítica	2%	
3	Public-Facing	Média	1%	
4	Internet-Facing	Alta	40%	

Defina a priorização, sendo 1 maior prioridade e 4 menor prioridade.

DESAFIO: RESPOSTA

Uma ferramenta que analisa vulnerabilidades em bibliotecas incorporadas ao software detectou um conjunto de vulnerabilidades. A área de segurança sumarizou as vulnerabilidades como se segue:

ID	AMBIENTE	CVSS	EPSS	PRIORIZAÇÃO
1	Frontend	Média	97%	1
2	Backend	Crítica	2%	4
3	Public-Facing	Média	1%	3
4	Internet-Facing	Alta	40%	2

Defina a priorização, sendo 1 maior prioridade e 4 menor prioridade.

PUC Minas

DESAFIO II
(Home Work)

DESAFIO: O PROBLEMA

Um conjunto de investidores está pretendendo investir em uma Startup para oferecer um produto Web para clientes corporativos com foco na triagem de candidatos a emprego, minimizando horas de trabalhado de Head Hunters. A preocupação dos investidores é com foco na qualidade e a satisfação dos clientes corporativos e usuários. Os investidores também estão preocupados com a segurança do produto e com leis de privacidade de dados, como a LGPD. Em uma análise do estado atual da Startup, foi observado durante uma avaliação de negócio que há sérias deficiências na relação com o cliente corporativo, demora na entrega, há silos entre as principais áreas que desenvolvem e sustentam o produto, muito trabalho manual para a entrega do produto e suas features constituintes, mudanças ocorrem no produto e infraestrutura causando interrupções sérias, não há visibilidade de vulnerabilidades no produto, e ainda, problemas de disponibilidade ocorrem frequentemente, e a equipe é muito reativa.

DESAFIO: O CENÁRIO

Você foi contratado para propor melhorias e correções no processo de desenvolvimento e sustentação do produto antes dos investidores de fato decidirem realizar o investimento. Apresente suas considerações gerais que podem trazer mudanças concretas para que esta Startup receba o investimento esperado.

Você deve considerar os aspectos de **Pessoas**, **Processos** e **Tecnologia**, com foco também na **segurança**.

DESAFIOS: PONTOS DE MELHORIA

Relação Cliente x Desenvolvedores

Demora na Entrega das Features e Mudanças do Produto

Silos em Áreas-Chave do Processo

Trabalho Manual

Mudanças Afetam o Produto

Causa de Indisponibilidade e Trabalho **Reativo**

PUC Minas